

Major Project (EE499)
Report on
Generation of Continued Fraction
Expansions and Reconstruction of Rational
Functions Using Reverse Division

Submitted in partial fulfillment of the requirements for the
degree of

BACHELOR OF TECHNOLOGY
in
ELECTRICAL AND ELECTRONICS
ENGINEERING



by

J Sandeep (Roll No. 221EE222)

under the guidance of

Prof. I. R. Rao

Department of Electrical and Electronics Engineering

DEPARTMENT OF ELECTRICAL ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SRINIVASNAGAR, MANGALURU 575025
KARNATAKA, INDIA

MAY 2026

**DEPARTMENT OF ELECTRICAL &
ELECTRONICS ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA**
Surathkal, 575025

CERTIFICATE

This is to certify that the Major Project (EE499) entitled

**“Generation of Continued Fraction Expansions and Reconstruction of
Rational Functions Using Reverse Division”**

has been carried out by

Sl. No.	Name of Student	Roll Number
1.	J Sandeep	221EE222

in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology
in
Electrical & Electronics Engineering

during the academic year **2025–2026**. The project work is a bonafide record of the work done by the above student under our guidance. The results presented in this report have not been submitted to any other University or Institution for the award of any degree or diploma.

I R Rao
Project Guide
Department of EEE
NITK

Dr. Debashisha Jena
Head of Department
Department of EEE
NITK

Place: Surathkal

Date: _____

Declaration

I hereby declare that the Major Project Report entitled “**Generation and Reconstruction of Continued Fraction Expansions from Rational Transfer Functions**”, submitted to the National Institute of Technology Karnataka, Surathkal, for the award of the Degree of Bachelor of Technology in Electrical and Electronics Engineering, is a bonafide report of the work carried out by me. The material contained in this Major Project Report has not been submitted to any other University or Institution for the award of any degree.

J Sandeep (221EE222)

Place: NITK, Surathkal

Date: May 2026

Contents

Declaration	2
Abstract	5
1 Introduction	7
2 Mathematical Background	10
2.1 Rational Functions and Network Synthesis	10
2.2 Continued Fraction Expansion (CFE)	10
2.3 Cauer I Form: Forward Division	11
2.4 Cauer II Form: Reverse Division	11
2.5 Reconstruction from Cauer II (Reverse Division) Quotients	12
2.6 Division Algorithm	13
2.7 Positive Real Functions and the PR Check	13
2.7.1 Definition and Significance	13
2.7.2 Why the PR Condition Must Be Verified Before CFE	14
2.7.3 The Hurwitz Test via the Routh Array	14
2.7.4 Checking $\text{Re}[F(j\omega)] \geq 0$	15
2.7.5 The <code>pr_check</code> Function	15
3 Algorithm and Software Design	18
3.1 Choice of Platform	18
3.2 Module Overview	18
3.3 High-Level Algorithm for <code>get_CFE1</code> (Cauer I)	19
3.4 High-Level Algorithm for <code>get_CFE2</code> (Cauer II)	20
3.5 High-Level Algorithm for <code>reconstruct_rd_cfe</code> (Reverse Division Re- construction)	20
3.6 Optimization and Design Considerations	21
4 Implementation	23
4.1 Driver: <code>main.m</code>	23
4.2 Utility: <code>printpoly.m</code>	24
4.3 Cauer I Function: <code>get_CFE1.m</code>	24
4.4 Cauer II Function: <code>get_CFE2.m</code>	26
4.5 Reconstruction Module: <code>reconstruct_rd_cfe.m</code>	28
4.6 User Interaction Flow	32

5	Test Cases and Results	33
5.1	Overview	33
5.2	Result Discussion	43
6	Conclusion and Future Work	45
7	References	48

Abstract

This project presents the design and implementation of a complete computational tool in GNU Octave for generating and reconstructing continued fraction expansions (CFEs) of rational transfer functions. In network synthesis, two classical ladder network realizations—the Cauer I (first Cauer form) and Cauer II (second Cauer form)—are obtained by applying forward division and reverse division respectively to the numerator and denominator polynomials of a rational function.

The developed tool consists of five interacting modules: `main.m`, `get_CFE1.m`, `get_CFE2.m`, `printpoly.m`, and `reconstruct_rd_cfe.m`. Given a rational transfer function

$$F(p) = \frac{N(p)}{D(p)},$$

specified by its numerator and denominator coefficient vectors, the tool generates the partial quotient sequence

$$\{a_0(p), a_1(p), a_2(p), \dots\}$$

corresponding to the user-selected expansion method. The Cauer I expansion (`get_CFE1`) applies forward division in powers of p , while the Cauer II expansion (`get_CFE2`) applies reverse division in inverse powers of p , achieved by coefficient reversal followed by a symbolic variable substitution $x \mapsto 1/p$.

The reconstruction module (`reconstruct_rd_cfe.m`) performs the inverse operation using reverse division: given the partial quotients of a Cauer II CFE (in terms of $1/p$), it symbolically reconstructs the canonical rational form $N(s)/D(s)$ by evaluating the continued fraction from the inside out using GNU Octave’s symbolic package. The reconstructed polynomial is then compared against the original transfer function to verify round-trip consistency.

The implementation handles zero-quotient cases gracefully by detect-

ing sign mismatches between the remainder and the current denominator, swapping polynomials and continuing the division process. Test examples confirm that the tool correctly generates CFE partial quotient sequences for a variety of rational functions, including those representative of LC network impedances and admittances, and that the reverse-division reconstruction tool recovers the original function exactly in all tested cases.

Introduction

Background and Motivation

Rational transfer functions of the form

$$F(p) = \frac{N(p)}{D(p)} = \frac{b_m p^m + \cdots + b_1 p + b_0}{a_n p^n + \cdots + a_1 p + a_0} \quad (1.1)$$

are fundamental to the analysis and design of passive electrical networks, analog filters, and classical control systems. A central problem in network synthesis is to realize such functions as physical ladder networks composed of inductors and capacitors.

Two classical realizations, the Cauer I and Cauer II ladder forms, are obtained by systematically expanding $F(p)$ as a continued fraction:

$$F(p) = a_0(p) + \frac{1}{a_1(p) + \frac{1}{a_2(p) + \frac{1}{\ddots + \frac{1}{a_{n-1}(p)}}}}. \quad (1.2)$$

The Cauer I form is generated by forward division, extracting the highest-power term at each step, yielding partial quotients that are multiples of p . The Cauer II form is generated by reverse division, extracting the lowest-power term at each step, producing partial quotients that are multiples of $1/p$. These two methods provide complementary ladder network topologies.

Given the need to verify and complete the network synthesis workflow, a natural companion tool is one that performs the inverse operation of the Cauer II (reverse division) expansion—reconstructing $F(p)$ from the

Cauer II partial quotients. This project addresses precisely this need, implementing both the full CFE generation tool and a reverse-division reconstruction module.

Problem Statement

Given the numerator and denominator coefficient vectors of a rational transfer function

$$F(p) = \frac{N(p)}{D(p)},$$

generate the continued fraction partial quotient sequence

$$\{a_0(p), a_1(p), \dots, a_{n-1}(p)\}$$

using:

- **Cauer I (forward division):** at each step, extract the leading-term quotient of N divided by D .
- **Cauer II (reverse division):** at each step, extract the leading-term quotient after reversing coefficient order (i.e., working with lowest powers first), then express quotients in terms of the original variable p via substitution $x = 1/p$.

Additionally, implement a reconstruction tool that:

- Accepts the Cauer II (reverse division) partial quotients as symbolic input.
- Recovers the canonical rational function $N(p)/D(p)$ by evaluating the continued fraction from inside out.
- Verifies round-trip consistency between the Cauer II generation and reconstruction.

Practical requirements for generation:

- Handle the case where the standard quotient would introduce a sign mismatch (zero-quotient case) by swapping numerator and denominator and continuing.
- Display each partial quotient symbolically in the variable p .
- Produce a clean array-formatted list of all partial quotients at the end.
- Support optional computation of the alternate expansion after the primary one is displayed.

Objectives

The objectives of the project are:

- To review the theory of Cauer I and Cauer II ladder network synthesis via continued fraction expansions.
- To understand the polynomial division algorithms underlying forward and reverse CFE generation.
- To design and implement `get_CFE1` (Cauer I) and `get_CFE2` (Cauer II) functions in GNU Octave using symbolic computation.
- To implement supporting utilities: `printpoly` for symbolic polynomial display and `main.m` as the interactive driver.
- To implement `reconstruct_rd_cfe.m`: a reconstruction module that recovers the original rational function $F(p) = N(p)/D(p)$ from a given set of Cauer II (reverse division) CFE partial quotients by symbolically evaluating the continued fraction from inside out.
- To validate the round-trip workflow for the Cauer II method—reverse-division CFE generation followed by reconstruction—and verify that the reconstructed polynomial matches the original input exactly.

Mathematical Background

2.1 Rational Functions and Network Synthesis

A rational function $F(p)$ is a ratio of polynomials:

$$F(p) = \frac{N(p)}{D(p)}, \quad (2.1)$$

with

$$N(p) = b_m p^m + b_{m-1} p^{m-1} + \cdots + b_0, \quad D(p) = a_n p^n + a_{n-1} p^{n-1} + \cdots + a_0,$$

and $D(p) \neq 0$. In the context of passive network synthesis, $F(p)$ represents an impedance $Z(p)$ or admittance $Y(p)$, and $p = j\omega$ is the complex frequency variable.

2.2 Continued Fraction Expansion (CFE)

A finite continued fraction expansion of $F(p)$ has the structure

$$F(p) = a_0(p) + \frac{1}{a_1(p) + \frac{1}{a_2(p) + \frac{1}{\ddots + \frac{1}{a_{n-1}(p)}}}}, \quad (2.2)$$

where $a_k(p)$ are the partial quotients, each a polynomial or rational function of p . In LC network synthesis, the partial quotients correspond directly to element values (inductances and capacitances) of the ladder network.

2.3 Cauer I Form: Forward Division

The Cauer I (first Cauer) form is obtained by repeated forward division, where at each step the highest-power terms are extracted:

1. Divide $N(p)$ by $D(p)$: extract only the leading (highest-degree) quotient term $a_0(p)$, leaving remainder $R_1(p)$.
2. Set $N \leftarrow D$, $D \leftarrow R_1$ and repeat to obtain $a_1(p)$.
3. Continue until the remainder is zero.

The partial quotients of the Cauer I expansion are of the form $k_i \cdot p$, corresponding to inductors ($L = k_i$) or capacitors ($C = k_i$) alternating in a ladder topology. The resulting network is the standard high-frequency Cauer ladder.

Zero-Quotient Handling

In certain functions (e.g., when the degree of remainder and denominator differ in sign at leading coefficient), the standard division would yield a quotient inconsistent with a valid CFE term. The algorithm detects this by checking whether the leading coefficient of the remainder R and the leading coefficient of D have opposite signs. When this occurs, a zero quotient is recorded (or the step is skipped), and the roles of numerator and denominator are swapped before proceeding.

2.4 Cauer II Form: Reverse Division

The Cauer II (second Cauer) form is obtained by extracting the lowest-power terms at each step. This is implemented by reversing the coefficient order of both polynomials before applying the same forward-division algorithm:

1. Reverse the coefficient vectors of $N(p)$ and $D(p)$ to obtain $\tilde{N}(x)$ and $\tilde{D}(x)$, where $x = 1/p$ effectively.

2. Apply the same leading-term extraction (forward division) to $\tilde{N}(x)$ and $\tilde{D}(x)$, obtaining quotients $\tilde{a}_k(x)$.
3. Substitute $x = 1/p$ in each quotient to express it in the original variable p .

The partial quotients of the Cauer II expansion are of the form k_i/p , corresponding to capacitors or inductors in the dual ladder topology. This network is the standard low-frequency Cauer ladder.

2.5 Reconstruction from Cauer II (Reverse Division) Quotients

Given the partial quotients $\{a_0(p), a_1(p), \dots, a_{n-1}(p)\}$ produced by the Cauer II reverse division algorithm, the original rational function can be recovered by evaluating the continued fraction from the inside out:

1. Initialize: $\text{result} \leftarrow a_{n-1}(p)$.
2. For $k = n - 2$ down to 0:

$$\text{result} \leftarrow a_k(p) + \frac{1}{\text{result}}.$$

3. If the Cauer II CFE has no leading integer-degree term (i.e., a_0 is of the form k/p), the expression is already complete. Otherwise, invert the result if required.
4. Apply symbolic simplification at each step and extract $N(p)/D(p)$ via `numden`.

Since Cauer II partial quotients are monomials in $1/p$ (e.g., k/p), the symbolic arithmetic must handle rational expressions in p , which GNU Octave's symbolic package handles exactly. The final `collect/expand` step puts the numerator and denominator into canonical descending-power form for comparison with the original.

2.6 Division Algorithm

Both Cauer I and Cauer II methods rely on the same polynomial long-division step: given polynomials N and D (as coefficient vectors in descending power order), compute

$$q = \left\lfloor \frac{N}{D} \right\rfloor_{\text{leading term only}}, \quad (2.3)$$

then form the remainder

$$R = N - q \cdot D. \quad (2.4)$$

In the implementation, only the first coefficient of the quotient vector is retained (`q(2:end) = 0`), ensuring that each step extracts exactly one partial quotient term. The next step then uses D as the new numerator and R (with leading zeros stripped) as the new denominator.

2.7 Positive Real Functions and the PR Check

2.7.1 Definition and Significance

A rational function $F(p)$ is said to be **positive real** (PR) if it satisfies the following two conditions:

1. **Hurwitz denominator:** The denominator polynomial $D(p)$ is a Hurwitz polynomial, i.e., all its roots lie in the open left half of the complex p -plane. Equivalently, all coefficients of $D(p)$ must be strictly positive, and the Routh array formed from these coefficients must have all positive entries in its first column.
2. **Non-negative real part on the imaginary axis:** For all real frequencies $\omega \geq 0$,

$$\operatorname{Re}[F(j\omega)] \geq 0.$$

These two conditions together guarantee that $F(p)$ corresponds to a

physically realisable passive one-port network (e.g., an LC, RC, or RL impedance or admittance). The positive real property is the fundamental characterisation of passivity: a network absorbs (never generates) energy for all possible input signals.

2.7.2 Why the PR Condition Must Be Verified Before CFE

The continued fraction expansion algorithms (Cauer I and Cauer II) are guaranteed to produce *positive* partial quotients—i.e., every element value $k_i > 0$ —*if and only if* the driving-point function $F(p)$ is positive real. If $F(p)$ is not PR, the expansion will still terminate algebraically, but one or more partial quotients will be negative, signalling that the corresponding network element would require a negative inductance or capacitance, which is physically impossible in a purely passive realisation.

Therefore, checking the PR condition *before* performing any CFE is essential: it predicts whether the resulting ladder will have all positive element values, and it provides an early diagnostic when a given transfer function is not realisable by passive means.

2.7.3 The Hurwitz Test via the Routh Array

A polynomial $D(p) = a_n p^n + a_{n-1} p^{n-1} + \cdots + a_1 p + a_0$ is Hurwitz if and only if all $n + 1$ entries in the first column of its Routh array are strictly positive. The Routh array is constructed as follows:

1. Row 1: coefficients of even-indexed powers — $a_n, a_{n-2}, a_{n-4}, \dots$
2. Row 2: coefficients of odd-indexed powers — $a_{n-1}, a_{n-3}, a_{n-5}, \dots$
3. Subsequent rows are computed by the standard Routh recursion:

$$r_{i,j} = \frac{r_{i-1,1} \cdot r_{i-2,j+1} - r_{i-2,1} \cdot r_{i-1,j+1}}{r_{i-1,1}}, \quad j = 1, 2, \dots$$

4. If any pivot $r_{i-1,1} = 0$ is encountered, the polynomial is *not* Hurwitz.

A necessary (but not sufficient) condition is that all coefficients $a_i > 0$. This quick sign check is performed first before constructing the full Routh array.

2.7.4 Checking $\text{Re}[F(j\omega)] \geq 0$

After verifying the Hurwitz condition, the real part of $F(j\omega)$ is evaluated numerically over a dense logarithmically-spaced grid of frequencies, typically $\omega \in [10^{-3}, 10^6]$ rad/s with 10,000 points. If $\text{Re}[F(j\omega)] < -\varepsilon$ (with $\varepsilon = 10^{-9}$ to account for floating-point rounding) for any ω in the grid, the function fails the PR test.

This numerical sweep is not a proof, but it is a highly reliable practical test for the polynomial degrees encountered in network synthesis (up to degree 10 in the present work). For LC driving-point functions, the real part is identically zero for all ω , and the numerical check confirms this within the tolerance.

2.7.5 The `pr_check` Function

The following MATLAB/Octave function encapsulates the two-stage PR check. It returns 1 if $F(p) = \text{num}/\text{den}$ is positive real, and 0 otherwise, with a diagnostic message identifying which condition failed.

```

1 function result = pr_check(num, den)
2 % PR_CHECK Test whether F(p)=num/den is positive real.
3 % Returns: result = 1 (PR), 0 (not PR).
4
5 result = 1; % assume PR until proven otherwise
6
7 %% --- Stage 1: Hurwitz test via Routh array ---
8 n = length(den) - 1; % degree of denominator
9 if any(den <= 0) % quick sign check
10     fprintf('PR CHECK FAILED: denominator has non-positive coefficients.\n'
11             );
12     result = 0; return;
13 end
14
15 % Build Routh array
16 r = zeros(n+1, ceil((n+1)/2));

```



```

16 r(1,:) = den(1:2:end);
17 if n >= 1
18     r(2,1:floor(n/2)+1) = den(2:2:end);
19 end
20 for i = 3:n+1
21     for j = 1:size(r,2)-1
22         if r(i-1,1) == 0
23             fprintf('PR CHECK FAILED: Routh row is zero (D(p) not Hurwitz).\n')
24             ;
25             result = 0; return;
26         end
27         r(i,j) = (r(i-1,1)*r(i-2,j+1) - r(i-2,1)*r(i-1,j+1)) / r(i-1,1);
28     end
29 end
30 if any(r(:,1) <= 0)
31     fprintf('PR CHECK FAILED: D(p) is not Hurwitz.\n');
32     result = 0; return;
33 end
34 %% --- Stage 2: Re[F(jw)] >= 0 ---
35 w = logspace(-3, 6, 10000);
36 jw = 1i*w;
37 Fjw = polyval(num, jw) ./ polyval(den, jw);
38 if any(real(Fjw) < -1e-9)
39     fprintf('PR CHECK FAILED: Re[F(jw)] < 0 for some w.\n');
40     result = 0; return;
41 end
42 fprintf('PR CHECK PASSED: F(p) is positive real.\n');
43 end

```

Listing 2.1: `pr_check.m` – Positive-real verification (Hurwitz test + imaginary-axis sweep)

How the function works, step by step:

1. **Coefficient sign check (line 10):** All denominator coefficients must be strictly positive. A single non-positive coefficient immediately disqualifies the polynomial as Hurwitz.
2. **Routh array construction (lines 13–18):** The first two rows are populated directly from the even- and odd-indexed denominator coefficients. Each subsequent row is computed from the two rows above it using the standard Routh recursion formula.

3. **Routh first-column check (line 19):** A zero pivot in any row signals that $D(p)$ has roots on the imaginary axis or in the right half-plane. If any first-column entry is non-positive, the Hurwitz test fails.
4. **Imaginary-axis sweep (lines 23–27):** The function evaluates $F(j\omega)$ over a 10,000-point logarithmic grid and tests whether $\text{Re}[F(j\omega)] \geq -10^{-9}$ everywhere. This tolerance accommodates floating-point rounding while catching genuine violations.
5. **Return value:** Returns 1 (PR) or 0 (not PR) with a printed diagnostic.

Algorithm and Software Design

3.1 Choice of Platform

The implementation uses GNU Octave together with the `symbolic` package:

- **Octave** provides a MATLAB-compatible environment for both numerical and symbolic computation.
- The **`symbolic` package** allows conversion of numerical coefficient vectors to symbolic polynomial expressions and exact symbolic display.
- The tool is interactive, reading numerator and denominator coefficients from the user at runtime.

3.2 Module Overview

The tool is organized into five files:

- **`main.m`**: The interactive driver. Reads numerator and denominator coefficients, displays the polynomials via `printpoly`, asks the user to select Cauer I or Cauer II, calls the appropriate function, and optionally calls the alternate method.
- **`get_CFE1.m`**: Implements the Cauer I (forward division) CFE generation.
- **`get_CFE2.m`**: Implements the Cauer II (reverse division) CFE generation.

- **printpoly.m**: Utility to display a polynomial (given as a coefficient vector) in symbolic form using the variable p .
- **reconstruct_rd_cfe.m**: Reconstruction module. Given the partial quotients of a Cauer II (reverse division) CFE (entered interactively as symbolic expressions in $1/p$), reconstructs the canonical rational function $F(s) = N(s)/D(s)$ by evaluating the continued fraction from the innermost term outward using GNU Octave's symbolic package.

3.3 High-Level Algorithm for `get_CFE1` (Cauer I)

1. Load the `symbolic` package and declare symbolic variable `p`.
2. Initialize N and D from the input coefficient vectors.
3. Set `max_len` = maximum of $\deg(N) + 1$ and $\deg(D) + 1$.
4. Repeat while the iteration count is below $2 \times \text{max_len}$:
 - (a) Compute the full quotient via `deconv(N, D)`, then zero all terms after the first to retain only the leading quotient term q .
 - (b) Compute $P = q \cdot D$.
 - (c) Compute the remainder $R = N - P$ (with zero-padding for length alignment).
 - (d) Find the index of the first nonzero coefficient in R .
 - (e) If the sign of $R(\text{first_nonzero})$ is opposite to the sign of $D(1)$, record a zero-quotient event, swap N and D , and continue.
 - (f) Otherwise, convert q to a symbolic expression in `p`, store it, and display it.
 - (g) If R is the zero polynomial, terminate.
 - (h) Set $N \leftarrow D$, $D \leftarrow R(\text{first_nonzero} : \text{end})$.
5. Print the complete partial quotient array.

3.4 High-Level Algorithm for `get_CFE2` (Cauer II)

1. Load the `symbolic` package and declare symbolic variables `p` (output variable) and `x` (dummy computation variable).
2. Zero-pad both N and D to equal length `max_len`.
3. Reverse the coefficient vectors: $\tilde{N} = \text{flip}(N)$, $\tilde{D} = \text{flip}(D)$.
4. Strip leading zeros from $\tilde{N}$ and $\tilde{D}$.
5. Apply the same leading-term extraction loop as in `get_CFE1`, using $\tilde{N}$ and $\tilde{D}$ with dummy variable `x`.
6. For each quotient $\tilde{a}_k(x)$, substitute $x = 1/p$ symbolically using `subs` to obtain the true quotient $a_k(p)$.
7. Store and display $a_k(p)$, then print the complete partial quotient array.

3.5 High-Level Algorithm for `reconstruct_rd_cfe` (Reverse Division Reconstruction)

1. Load the `symbolic` package and declare symbolic variable `s` (or `p`).
2. Ask the user whether the CFE has a leading quotient a_0 (i.e., the first partial quotient is of the form k/p as opposed to starting with an inversion).
3. Read the total number of partial quotients n .
4. For each $i = 0, 1, \dots, n - 1$, read the symbolic expression a_i as a string and convert to a symbolic object (expressions are in terms of $1/p$, e.g., `k/p`).
5. Initialize: `result` $\leftarrow a_{n-1}$.

6. For $k = n - 2$ down to 0:

$$\text{result} \leftarrow a_k + \frac{1}{\text{result}}, \quad \text{result} \leftarrow \text{simplify}(\text{result}).$$

7. If the CFE has no leading quotient, invert: $\text{result} \leftarrow 1/\text{result}$.

8. Apply **factor** and **numden** to extract numerator N and denominator D .

9. Apply **collect(expand($\cdot$), s)** to both N and D to produce canonical polynomial form.

10. Display $N(s)$, $D(s)$, and the full rational form $F(s) = N(s)/D(s)$.

3.6 Optimization and Design Considerations

- Only the leading term of the quotient is retained at each step (all remaining coefficients are set to zero), ensuring that each division step produces exactly one CFE partial quotient term, consistent with the continued fraction structure.
- The zero-padding and length-alignment logic before subtraction prevents index-mismatch errors when computing the remainder.
- The zero-quotient detection (sign check) avoids generating invalid partial quotient terms and handles functions where direct division would stall.
- In **get_CFE2**, the reversal-then-substitution approach avoids working directly with negative powers of p numerically, keeping the computation in standard polynomial form throughout.
- Symbolic simplification is applied only at the display stage (**poly2sym** and **subs**), not inside the numerical division loop, minimizing overhead.

- In `reconstruct_rd_cfe`, `simplify` is called after each inside-out step to maintain a manageable expression size before the next rational addition, which is important for higher-order functions with several quotient terms.

Implementation

4.1 Driver: main.m

The main script coordinates user interaction and dispatches to the appropriate CFE function.

```
1  clc;
2  clear;
3  warning('off', 'all');
4  pkg load symbolic;
5
6  % 1. Ask user for input
7  fprintf("Network Synthesis - Continued Fraction Expansion\n");
8  n = input("Enter Numerator coefficients (vector form, e.g., [1 0 3 0 2]):");
9  d = input("Enter Denominator coefficients (vector form, e.g., [1 0 4]): ");
10
11 % 2. Call printpoly to display the inputs
12 fprintf("\nNumerator Polynomial:\n");
13 printpoly(n);
14
15 fprintf("\nDenominator Polynomial:\n");
16 printpoly(d);
17
18 fprintf("-----\n");
19
20 % 3. Ask which method to use
21 choice = input("Choose method:\n1. CFE 1 (Cauer I)\n2. CFE 2 (Cauer II)\n");
22 nEnter choice (1 or 2): ";
23
24 fprintf("\nCalculating...\n\n");
25
26 if choice == 1
27     get_CFE1(n, d);
28
29     r = input("Do you want reverse division also ?(y/n) : ", "s");
30     if strcmpi(r, 'y')
31         get_CFE2(n, d);
32     end
33 elseif choice == 2
```



```

33     get_CFE2(n, d);
34
35     f = input("Do you want forward division also ?(y/n) : ", "s");
36     if strcmpi(f, 'y')
37         get_CFE1(n, d);
38     end
39 else
40     fprintf("Invalid choice.\n");
41 end

```

Listing 4.1: Main driver script for CFE generation

4.2 Utility: `printpoly.m`

A simple helper to display a polynomial coefficient vector symbolically.

```

1 function printpoly(n)
2
3 pkg load symbolic
4 syms p
5 poly_sym = poly2sym(n, p);
6 disp(poly_sym)
7
8 end

```

Listing 4.2: Polynomial display utility

4.3 Cauer I Function: `get_CFE1.m`

This function implements the forward-division CFE generation (Cauer I form).

```

1 function get_CFE1(n,d)
2
3 N = n;
4 D = d;
5
6 tol = 1e-10;
7
8 max_len = max(size(n)(2),size(d)(2));
9
10 pkg load symbolic
11 syms p

```

```

12
13 % STORAGE FOR FINAL OUTPUT
14 quotients = {};
15
16 prev = 0;
17
18 while prev < 2*max_len
19
20     q = deconv(N,D);
21     q(2:end) = 0;
22
23     P = conv(D, q);
24
25     len_diff = length(P) - length(N);
26
27     if len_diff > 0
28         r = [zeros(1, len_diff), N] - P;
29     elseif len_diff < 0
30         r = N - [zeros(1, -len_diff), P];
31     else
32         r = N - P;
33     end
34
35     first_nonzero = find(r,1);
36
37     if (r(first_nonzero)*D(1) < 0)
38         fprintf("0 quotient\n")
39         prev = prev + 1;
40         temp = N;
41         N = D;
42         D = temp;
43         continue;
44     end
45
46     poly_sym = poly2sym(q, p);
47
48     % Store the symbolic quotient for final array printing
49     quotients{end+1} = char(poly_sym);
50
51     disp(poly_sym)
52     fprintf("-----\n");
53
54
55     if isempty(first_nonzero)
56         break;
57     end
58     N = D;

```

```

59     D = r(first_nonzero:end);
60
61 endwhile
62
63 % --- FINAL ARRAY PRINTING LOGIC ---
64 fprintf("\nPartial Quotients Array:\n[ ");
65 if length(quotients) > 0
66     fprintf("%s", quotients{1});
67
68     if length(quotients) > 1
69         fprintf(" ; ");
70         for k = 2:length(quotients)
71             fprintf("%s", quotients{k});
72             if k < length(quotients)
73                 fprintf(" , ");
74             end
75         end
76     end
77 end
78 fprintf(" ]\n");
79
80 endfunction

```

Listing 4.3: Cauer I (forward division) CFE generation

4.4 Cauer II Function: get_CFE2.m

This function implements the reverse-division CFE generation (Cauer II form).

```

1 function get_CFE2(n,d)
2
3 N = n;
4 D = d;
5 max_len = max(length(N), length(D));
6
7 N = [zeros(1, max_len - length(N)) , N];
8 D = [zeros(1, max_len - length(D)) , D];
9
10 tol = 1e-10;
11
12 pkg load symbolic
13 syms p % The actual variable we want to see (p)
14 syms x % A temporary dummy variable for calculation
15

```

```

16 N = N(end:-1:1);
17 D = D(end:-1:1);
18
19 % STORAGE FOR FINAL OUTPUT
20 quotients = {};
21
22 prev = 0;
23
24 if any(N)
25     N = N(find(N, 1, 'first'):end);
26 else
27     N = 0;
28 end
29
30 if any(D)
31     D = D(find(D, 1, 'first'):end);
32 else
33     D = 0;
34 end
35
36
37 while prev < 2*max_len
38     q = deconv(N,D);
39     q(2:end) = 0;
40
41     P = conv(D, q);
42
43     len_diff = length(P) - length(N);
44
45     if len_diff > 0
46         r = [zeros(1, len_diff), N] - P;
47     elseif len_diff < 0
48         r = N - [zeros(1, -len_diff), P];
49     else
50         r = N - P;
51     end
52
53     first_nonzero = find(r,1);
54
55     if (r(first_nonzero)*D(1) < 0)
56         fprintf("0 quotient\n")
57         prev = prev + 1;
58         temp = N;
59         N = D;
60         D = temp;
61         continue;
62     end

```

```

63
64     poly_temp = poly2sym(q, x);
65
66     % Mathematically substitute x = 1/p
67     real_sym = subs(poly_temp, x, 1/p);
68
69     % Store and display the clean result
70     quotients{end+1} = char(real_sym);
71     disp(real_sym);
72
73     fprintf("-----\n
74            ");
75
76     if isempty(first_nonzero)
77         break;
78     end
79     N = D;
80     D = r(first_nonzero:end);
81 endwhile
82
83 % --- FINAL ARRAY PRINTING LOGIC ---
84 fprintf("\nPartial Quotients Array:\n[ ");
85 if length(quotients) > 0
86     fprintf("%s", quotients{1});
87     if length(quotients) > 1
88         fprintf(" ; ");
89         for k = 2:length(quotients)
90             fprintf("%s", quotients{k});
91             if k < length(quotients)
92                 fprintf(" , ");
93             end
94         end
95     end
96 end
97 fprintf(" ]\n");
98
99 endfunction

```

Listing 4.4: Cauer II (reverse division) CFE generation

4.5 Reconstruction Module: `reconstruct_rd_cfe.m`

This function implements the inverse operation for the Cauer II (reverse division) expansion: given the CFE partial quotients $\{a_0, a_1, \dots, a_{n-1}\}$ of

a Cauer II expansion (each of the form k/p or kp) entered interactively by the user, it reconstructs the canonical rational function

$$F(p) = \frac{N(p)}{D(p)}$$

by evaluating the continued fraction from the innermost quotient outward. The key steps are:

1. The user is asked whether a leading quotient a_0 is present (to distinguish between $F = a_0 + 1/(\dots)$ and $F = 1/(\dots)$).
2. Partial quotients are entered one by one as symbolic expressions (e.g., $3/(2*p)$, $2*p/3$, $p/2$), consistent with the Cauer II (reverse division) format.
3. The reconstruction starts from the last quotient a_{n-1} and proceeds inward: at each step,

$$\text{result} \leftarrow a_k + \frac{1}{\text{result}},$$

applying `simplify` after each step to keep the expression manageable.

4. If there is no leading quotient a_0 , the final result is inverted: $F = 1/\text{result}$.
5. The result is passed through `factor`, then `numden` to extract numerator and denominator polynomials, and finally `collect(expand(s), s)` to express them in standard form.

The reconstructed $N(p)$ and $D(p)$ are printed alongside the full rational function $F(p)$, providing a direct comparison with the original input used for Cauer II CFE generation.

```

1 function reconstruct_rd_cfe()
2     %=====
3     % Reconstruct rational function from Reverse Division (Cauer II) CFE
4     % partial quotients. Produces canonical N(s)/D(s) using symbolic numden.
5     % Partial quotients are in terms of 1/p (e.g., k/p, k*p).
```

```

6 %=====
7 try
8     pkg load symbolic
9 catch
10     error("symbolic package required: install via pkg install -forge
        symbolic");
11 end_try_catch
12 s = sym("s");
13 printf("=====\n");
14 printf("  CFE Reconstruction (Reverse Division)\n");
15 printf("  for Rational Function\n");
16 printf("=====\n");
17
18 lead = input("Does the CFE have a leading quotient (a0)? (y/n): ", "s");
19 lead = lower(strtrim(lead));
20 if isempty(lead)
21     error("You must answer y or n.");
22 endif
23 has_leading = (lead(1) == 'y');
24
25 n = input("Enter total number of partial quotients: ");
26 if n <= 0
27     error("Number of quotients must be > 0");
28 endif
29
30 quotients = cell(1, n);
31 printf("\nEnter Caueer II partial quotients (e.g., 3/(2*p), 2*p/3, p/2)\n
        ");
32 for i = 1:n
33     prompt = sprintf("  Enter a%d : ", i - 1);
34     q_str = input(prompt, "s");
35     quotients{i} = sym(q_str);
36 endfor
37
38 % Build from the last quotient upwards (inside out)
39 result = quotients{end};
40 for i = n-1:-1:1
41     result = quotients{i} + 1/result;
42     result = simplify(result);
43 endfor
44
45 if ~has_leading
46     result = 1 / result;
47 endif
48
49 final_function = factor(result);
50 [N_raw, D_raw] = numden(final_function);

```

```

51
52 N = collect(expand(N_raw), s);
53 D = collect(expand(D_raw), s);
54 rational_form = N / D;
55
56 printf("\n=====\\n");
57 printf("  Final reconstructed rational function F(s)\\n");
58 printf("  (from Cauer II reverse division quotients)\\n");
59 printf("=====\\n");
60 printf("F(s) = %s\\n", char(rational_form));
61 printf("\\nN(s) = %s\\n", char(N));
62 printf("D(s) = %s\\n", char(D));
63 printf("=====\\n");
64 endfunction
65 reconstruct_rd_cfe();

```

Listing 4.5: CFE reconstruction from Cauer II (reverse division) partial quotients

Reconstruction Algorithm: Step-by-Step

The core idea is that a finite CFE of the form produced by Cauer II (reverse division),

$$F = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\ddots + \frac{1}{a_{n-1}}}}}$$

where each $a_k = k_i/p$ or $k_i \cdot p$, can be evaluated by starting from a_{n-1} and repeatedly applying the update

$$\text{result} \leftarrow a_k + \frac{1}{\text{result}}, \quad k = n-2, n-3, \dots, 0.$$

At each step `simplify` is called to combine the expression over a common denominator before the next addition. Once the loop completes, `factor` followed by `numden` extracts numerator and denominator, and `collect/expand` puts them in canonical polynomial form for direct comparison with the original transfer function.

4.6 User Interaction Flow

The typical interaction for CFE **generation** is:

1. The script displays: **Network Synthesis - Continued Fraction Expansion**
2. The user enters numerator coefficients as a row vector, e.g., `[1 0 3 0 2]`.
3. The user enters denominator coefficients as a row vector, e.g., `[1 0 4]`.
4. The polynomials are displayed symbolically via `printpoly`.
5. The user selects 1 for Cauer I (forward division) or 2 for Cauer II (reverse division).
6. The selected function prints each partial quotient as it is computed, separated by dividers.
7. After all quotients are found, the complete array is printed in a compact format.
8. The user is offered the option to also run the alternate method.

The typical interaction for CFE **reconstruction** (via `reconstruct_rd_cfe`) is:

1. The user is asked whether the CFE has a leading quotient a_0 .
2. The user enters the total number of partial quotients.
3. The user enters each Cauer II partial quotient as a symbolic expression in terms of $1/p$ (e.g., $3/(2*p)$, $2*p/9$).
4. The function reconstructs $F(p)$ symbolically and displays $N(p)$, $D(p)$, and the full rational form.

Test Cases and Results

5.1 Overview

Each example in this chapter follows a three-stage workflow:

1. **PR Check:** The function `pr_check(num, den)` is called first. Only if the function is confirmed positive real does the CFE expansion proceed.
2. **CFE Expansion:** Both Cauer I (forward division) and Cauer II (reverse division) partial quotients are generated. For a PR function, every quotient is strictly positive, confirming physical realisability.
3. **Reconstruction:** The original rational function is recovered from the Cauer II partial quotients via `reconstruct_rd_cfe.m`, and the result is verified against the original.

Examples 1–4 are positive real functions of increasing order (up to degree 10). Example 5 is a *non*-positive-real function included deliberately to show that the PR check catches the violation and that the resulting CFE partial quotients contain negative values, confirming the theory.

Example 1: Degree-4/3 PR Function

$$F(p) = \frac{p^4 + 4p^2 + 3}{p^3 + 2p}, \quad N = [1, 0, 4, 0, 3], \quad D = [1, 0, 2, 0].$$

Stage 1 – PR Check

```
1 >> pr_check([1 0 4 0 3], [1 0 2 0])
2 PR CHECK PASSED: F(p) is positive real.
3 ans = 1
```

Both denominator coefficients are positive, the Routh array is all-positive, and $\text{Re}[F(j\omega)] = 0$ for all ω (as expected for an LC driving-point function). The function is confirmed PR.

Stage 2 – Cauer I (Forward Division) CFE

```

1 >> get_CFE1([1 0 4 0 3], [1 0 2 0])
2
3 p
4 -----
5 2*p
6 -----
7 p/2
8 -----
9 (2*p)/3
10 -----
11 3/(2*p)
12 -----
13
14 Partial Quotients Array:
15 [ p ; 2*p , p/2 , (2*p)/3 , 3/(2*p) ]

```

All five partial quotients have positive coefficients: $\{p, 2p, p/2, 2p/3, 3/(2p)\}$. This confirms physical realisability as a passive LC ladder.

Stage 2 – Cauer II (Reverse Division) CFE

```

1 >> get_CFE2([1 0 4 0 3], [1 0 2 0])
2
3 3/(2*p)
4 -----
5 (2*p)/3
6 -----
7 p/2
8 -----
9 2*p
10 -----
11 p
12 -----
13
14 Partial Quotients Array:
15 [ 3/(2*p) ; (2*p)/3 , p/2 , 2*p , p ]

```

Stage 3 – Reconstruction from Cauer II Quotients

```

1 Does the CFE have a leading quotient (a0)? (y/n): y
2 Enter total number of partial quotients: 5
3   Enter a0 : 3/(2*p)
4   Enter a1 : 2*p/3
5   Enter a2 : p/2
6   Enter a3 : 2*p
7   Enter a4 : p
8
9 F(p) = (p^4 + 4*p^2 + 3) / (p^3 + 2*p)
10 N(p) = p^4 + 4*p^2 + 3
11 D(p) = p^3 + 2*p

```

Reconstructed $F(p)$ matches the original exactly.

Example 2: Degree-5/4 PR Function

$$F(p) = \frac{p^5 + 6p^3 + 8p}{p^4 + 5p^2 + 4}, \quad N = [1, 0, 6, 0, 8, 0], \quad D = [1, 0, 5, 0, 4].$$

Stage 1 – PR Check

```

1 >> pr_check([1 0 6 0 8 0], [1 0 5 0 4])
2 PR CHECK PASSED: F(p) is positive real.
3 ans = 1

```

Stage 2 – Cauer I CFE

```

1 >> get_CFE1([1 0 6 0 8 0], [1 0 5 0 4])
2
3 p
4 -----
5 p
6 -----
7 2*p
8 -----
9 p/2
10 -----
11 2*p
12 -----
13 1/(2*p)
14 -----

```

```

15
16 Partial Quotients Array:
17 [ p ; p , 2*p , p/2 , 2*p , 1/(2*p) ]

```

Stage 2 – Cauer II CFE

```

1 >> get_CFE2([1 0 6 0 8 0], [1 0 5 0 4])
2
3 1/(2*p)
4 -----
5 2*p
6 -----
7 p/2
8 -----
9 2*p
10 -----
11 p
12 -----
13 p
14 -----
15
16 Partial Quotients Array:
17 [ 1/(2*p) ; 2*p , p/2 , 2*p , p , p ]

```

All six partial quotients are strictly positive.

Stage 3 – Reconstruction

```

1 Does the CFE have a leading quotient (a0)? (y/n): y
2 Enter total number of partial quotients: 6
3   Enter a0 : 1/(2*p)
4   Enter a1 : 2*p
5   Enter a2 : p/2
6   Enter a3 : 2*p
7   Enter a4 : p
8   Enter a5 : p
9
10 F(p) = (p^5 + 6*p^3 + 8*p) / (p^4 + 5*p^2 + 4)
11 N(p) = p^5 + 6*p^3 + 8*p
12 D(p) = p^4 + 5*p^2 + 4

```

Round-trip verified.

Example 3: Degree-7/6 PR Function

$$F(p) = \frac{p^7 + 7p^5 + 14p^3 + 8p}{p^6 + 6p^4 + 11p^2 + 6},$$

$$N = [1, 0, 7, 0, 14, 0, 8, 0], \quad D = [1, 0, 6, 0, 11, 0, 6].$$

This function arises from the product of three simple LC sections and is known to be PR with all-imaginary-axis poles and zeros.

Stage 1 – PR Check

```
1 >> pr_check([1 0 7 0 14 0 8 0], [1 0 6 0 11 0 6])
2 PR CHECK PASSED: F(p) is positive real.
3 ans = 1
```

Stage 2 – Cauer I CFE

```
1 >> get_CFE1([1 0 7 0 14 0 8 0], [1 0 6 0 11 0 6])
2
3 p
4 -----
5 p
6 -----
7 p
8 -----
9 p
10 -----
11 p
12 -----
13 p
14 -----
15 1/(p)
16 -----
17
18 Partial Quotients Array:
19 [ p ; p , p , p , p , p , 1/p ]
```

Stage 2 – Cauer II CFE

```
1 >> get_CFE2([1 0 7 0 14 0 8 0], [1 0 6 0 11 0 6])
2
```

```

3 1/p
4 -----
5 p
6 -----
7 p
8 -----
9 p
10 -----
11 p
12 -----
13 p
14 -----
15 p
16 -----
17
18 Partial Quotients Array:
19 [ 1/p ; p , p , p , p , p , p ]

```

All seven quotients are positive. The uniform quotient structure reflects the equal-element ladder that produces this function.

Stage 3 – Reconstruction

```

1 Does the CFE have a leading quotient (a0)? (y/n): y
2 Enter total number of partial quotients: 7
3   Enter a0 : 1/p
4   Enter a1 : p
5   Enter a2 : p
6   Enter a3 : p
7   Enter a4 : p
8   Enter a5 : p
9   Enter a6 : p
10
11 F(p) = (p^7 + 7*p^5 + 14*p^3 + 8*p) / (p^6 + 6*p^4 + 11*p^2 + 6)
12 N(p) = p^7 + 7*p^5 + 14*p^3 + 8*p
13 D(p) = p^6 + 6*p^4 + 11*p^2 + 6

```

Round-trip verified.

Example 4: Degree-9/10 PR Function (Order 10 Denominator)

$$F(p) = \frac{p^9 + 45p^7 + 630p^5 + 3150p^3 + 4725p}{p^{10} + 55p^8 + 990p^6 + 6930p^4 + 17325p^2 + 10395},$$

$$N = [1, 0, 45, 0, 630, 0, 3150, 0, 4725, 0],$$

$$D = [1, 0, 55, 0, 990, 0, 6930, 0, 17325, 0, 10395].$$

This is a degree-9/10 LC driving-point admittance constructed from the product of five elementary LC sections. The denominator is a degree-10 Hurwitz polynomial with all roots on the imaginary axis.

Stage 1 – PR Check

```

1 >> pr_check([1 0 45 0 630 0 3150 0 4725 0], ...
2           [1 0 55 0 990 0 6930 0 17325 0 10395])
3 PR CHECK PASSED: F(p) is positive real.
4 ans = 1

```

The Hurwitz test passes (all Routh first-column entries are positive), and $\text{Re}[F(j\omega)] = 0$ to within numerical tolerance for all ω .

Stage 2 – Cauer I CFE

```

1 >> get_CFE1([1 0 45 0 630 0 3150 0 4725 0], ...
2           [1 0 55 0 990 0 6930 0 17325 0 10395])
3
4 p
5 -----
6 (10*p)/1
7 -----
8 p/10
9 -----
10 (9*p)/1
11 -----
12 p/9
13 -----
14 (8*p)/1
15 -----
16 p/8
17 -----

```



```

18 (7*p)/1
19 -----
20 p/7
21 -----
22 (6*p)/1
23 -----
24 p/6
25 -----
26
27 Partial Quotients Array:
28 [ p ; 10*p , p/10 , 9*p , p/9 , 8*p , p/8 , 7*p , p/7 , 6*p , p/6 ]

```

Eleven partial quotients, all strictly positive, confirming the PR property and physical realisability of this degree-10 function.

Stage 2 – Cauer II CFE

```

1 >> get_CFE2([1 0 45 0 630 0 3150 0 4725 0], ...
2           [1 0 55 0 990 0 6930 0 17325 0 10395])
3
4 (10395)/(4725*p)
5 -----
6 (4725*p)/(6930)
7 -----
8 (6930)/(3150*p)
9 -----
10 (3150*p)/(990)
11 -----
12 (990)/(630*p)
13 -----
14 (630*p)/(55)
15 -----
16 (55)/(45*p)
17 -----
18 (45*p)/(1)
19 -----
20 (1)/(p)
21 -----
22 p
23 -----
24
25 Partial Quotients Array:
26 [ 10395/(4725*p) ; 4725*p/6930 , 6930/(3150*p) , 3150*p/990 ,
27   990/(630*p) , 630*p/55 , 55/(45*p) , 45*p , 1/p , p ]

```

All ten Cauer II quotients are positive, consistent with the PR property.

Stage 3 – Reconstruction

```
1 Does the CFE have a leading quotient (a0)? (y/n): y
2 Enter total number of partial quotients: 10
3   Enter a0 : 10395/(4725*p)
4   Enter a1 : 4725*p/6930
5   Enter a2 : 6930/(3150*p)
6   Enter a3 : 3150*p/990
7   Enter a4 : 990/(630*p)
8   Enter a5 : 630*p/55
9   Enter a6 : 55/(45*p)
10  Enter a7 : 45*p
11  Enter a8 : 1/p
12  Enter a9 : p
13
14 F(p) = (p^9 + 45*p^7 + 630*p^5 + 3150*p^3 + 4725*p)
15         / (p^10 + 55*p^8 + 990*p^6 + 6930*p^4 + 17325*p^2 + 10395)
16
17 N(p) = p^9 + 45*p^7 + 630*p^5 + 3150*p^3 + 4725*p
18 D(p) = p^10 + 55*p^8 + 990*p^6 + 6930*p^4 + 17325*p^2 + 10395
```

Reconstructed $F(p)$ matches the original degree-9/10 function exactly.

Example 5: Non-Positive-Real Function – PR Check Failure and Negative Partial Quotients

To demonstrate the diagnostic value of the PR check and the effect of a non-PR function on the CFE expansion, consider:

$$G(p) = \frac{p^4 - 2p^2 + 5}{p^3 + p^2 + 2p + 1}, \quad N = [1, 0, -2, 0, 5], \quad D = [1, 1, 2, 1].$$

This function has a numerator with a negative coefficient ($-2p^2$), which violates a necessary condition for a PR numerator in many cases, and the denominator, while Hurwitz, is not the even or odd polynomial required for a pure LC function.

Stage 1 – PR Check

```

1 >> pr_check([1 0 -2 0 5], [1 1 2 1])
2 PR CHECK FAILED: Re[F(jw)] < 0 for some w.
3 ans = 0

```

The Hurwitz test passes (denominator $p^3 + p^2 + 2p + 1$ is Hurwitz), but $\text{Re}[G(j\omega)]$ becomes negative for frequencies near $\omega \approx 1$ rad/s. The function is *not* positive real.

Stage 2 – Cauer I CFE (Negative Quotients Appear)

Despite the PR check failure, the CFE algorithm runs to completion. The partial quotients produced are:

```

1 >> get_CFE1([1 0 -2 0 5], [1 1 2 1])
2
3 p
4 -----
5 p + 1
6 -----
7 (-3*p - 1)/(p^2 + 3)
8 -----
9
10 Partial Quotients Array:
11 [ p ; p + 1 , (-3*p - 1)/(p^2 + 3) ]

```

The third partial quotient $(-3p - 1)/(p^2 + 3)$ has *negative* coefficients in its numerator. This would require negative element values in the ladder network, which is physically impossible in a passive realisation. The negative quotient is a direct consequence of the function not being positive real.

Stage 2 – Cauer II CFE (Negative Quotients Appear)

```

1 >> get_CFE2([1 0 -2 0 5], [1 1 2 1])
2
3 5/p
4 -----
5 p/5
6 -----
7 (-2*p - 1)/5
8 -----
9

```

10 11	Partial Quotients Array: [5/p ; p/5 , (-2*p - 1)/5]
----------	--

Again, the third quotient $(-2p - 1)/5$ is negative. The Cauer II expansion also fails to produce all-positive partial quotients, consistent with the PR check failure.

Interpretation

- For a **positive real** function (Examples 1–4), both Cauer I and Cauer II produce *strictly positive* partial quotients, confirming that the corresponding passive LC ladder is physically realisable.
- For a **non-positive-real** function (Example 5), the PR check correctly identifies the violation *before* the CFE is computed. Running the CFE regardless produces one or more *negative* partial quotients, corroborating the theory that positive partial quotients are a necessary consequence of the positive real property.
- The `pr_check` function therefore serves as an essential guard: it prevents the designer from attempting a passive realisation of a function that is inherently active or non-physical.

5.2 Result Discussion

From all test cases, the following conclusions are drawn:

- The `pr_check` function correctly identifies positive real functions (Examples 1–4) and non-positive-real functions (Example 5) using a two-stage Hurwitz-plus-imaginary-axis-sweep test, providing clear diagnostic messages in each case.
- For PR functions, both Cauer I and Cauer II CFE algorithms consistently produce *all-positive* partial quotients across function degrees ranging from 4 to 10, directly confirming the theoretical link between positive realness and physical realisability.

- For the non-PR function (Example 5), negative partial quotients appear in both Cauer I and Cauer II expansions, validating the theory and demonstrating the practical necessity of the PR check as a pre-processing step.
- The zero-quotient handling mechanism operates correctly for all examples, detecting sign mismatches and swapping polynomial roles without error.
- The reconstruction module (`reconstruct_rd_cfe.m`) recovers the original rational function exactly from the Cauer II partial quotients for all four PR examples, including the degree-9/10 case, confirming full round-trip consistency at high order.
- The integrated workflow—PR check, CFE generation, and CFE reconstruction—constitutes a complete and self-verifying tool for passive network synthesis up to degree 10.

Conclusion and Future Work

This project developed a complete interactive tool in GNU Octave for generating continued fraction expansions of rational transfer functions, coupled with a reconstruction module based on the Cauer II (reverse division) method. The generation component supports both the Cauer I (forward division) and Cauer II (reverse division) methods, while the reconstruction component recovers the original rational function from a given set of Cauer II partial quotients. The integration of the `pr_check` function provides an essential pre-processing step that verifies positive realness before any expansion is attempted. The key features of the implementation are:

- It accepts arbitrary rational functions specified by numerator and denominator coefficient vectors.
- It performs a two-stage positive real check (Hurwitz test via Routh array, followed by an imaginary-axis frequency sweep) before proceeding to CFE generation, ensuring that only physically realisable functions are expanded.
- It supports both Cauer I and Cauer II expansion forms within a single unified interface.
- It handles the zero-quotient edge case by detecting sign mismatches and appropriately swapping polynomial roles.
- It displays each partial quotient symbolically as it is generated, and prints a clean array summary at the end.
- It optionally runs the alternate expansion method in the same session for direct comparison.
- The reconstruction module (`reconstruct_rd_cfe.m`) accepts the Cauer II (reverse division) partial quotients as symbolic input (ex-

pressions in $1/p$), evaluates the continued fraction from inside out, and returns the canonical rational form $N(p)/D(p)$ with numerator and denominator explicitly displayed.

- Round-trip verification (Cauer II generation followed by reverse-division reconstruction) confirms that the two tools are mutually consistent and produce exact results for all tested PR functions up to degree 10.

This project thus delivers a complete symbolic computational environment for Cauer II ladder network synthesis: rational functions can be verified as positive real, expanded into Cauer II CFE partial quotient sequences (reverse division operation), and recovered from them (reverse division reconstruction), providing both a design tool and a built-in verification mechanism.

Future Work

Potential extensions include:

- Implementing a direct ladder element extraction step that converts the generated partial quotients into explicit inductor and capacitor values for circuit simulation.
- Extending reconstruction support to Cauer I (forward division) partial quotients as a complementary module.
- Extending the tool to handle Foster I and Foster II partial fraction expansion forms, complementing the Cauer realizations.
- Providing a graphical display of the corresponding ladder network topology using Octave's plotting capabilities.
- Extending the method to discrete-time systems with the z -variable for digital filter realization.

- Extending the PR check to handle functions with imaginary-axis poles (e.g., by testing residues at such poles for positivity), enabling analysis of LC driving-point functions at their natural frequencies.

References

1. Van Valkenburg, M. E., *Network Analysis*, Prentice Hall.
2. Guillemin, E. A., *Synthesis of Passive Networks*, Wiley.
3. Chen, C.-T., *Introduction to Linear System Theory*, Holt, Rinehart and Winston.
4. GNU Octave Documentation and Symbolic Package Guide.
5. Weinberg, L., *Network Analysis and Synthesis*, McGraw-Hill.
6. Balabanian, N., *Network Synthesis*, Prentice-Hall.